# Deep Learning and its Applications to Pollen Analysis

Lecture 1: classification

**Shu Kong**

University of Macau
https://aimerykong.github.io/
skong@um.edu.mo

43rd Mid-Continent Paleobotanical Colloquium
Panama 2026

# Roadmap

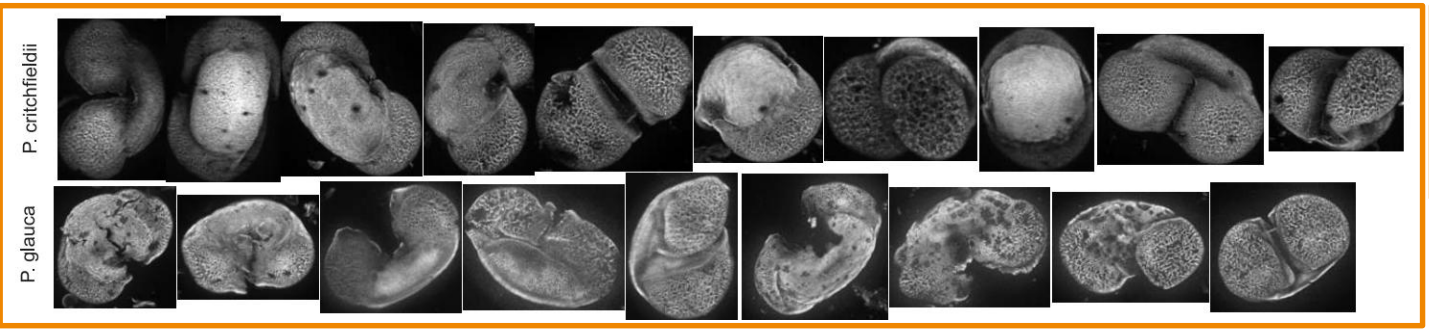
**Toolboxes**:

- platform: Python, Jupyter Notebook
- basics: sklearn, skimage, numpy, scipy, matplotlib
- for deep learning: PyTorch
- Teaching materials are available at https://github.com/aimerykong/MPC2026-tutorial

| Lecture | Topics |
|---|---|
| 1. Introduction | machine learning, computer vision, deep learning, artificial intelligence |
| 2. Basics via Classification | Nearest Neighbor, Perceptron, Logistic Regression, cross entropy, feature, gradient descent, etc. |
| 3. Deep Neural Networks | Multi-Layer Perceptron, feature learning, backpropagation, CNN, Transformer |
| 4. Hands-On Practice: Basics | pollen data preparation, visualization, nearest neighbor, linear classifier, CNN |
| 5. Segmentation and Detection | semantic segmentation, instance segmentation, U-Net, pixel grouping, mean-shift |
| 6. Foundation Models | CLIP, Segment Anything Model, DINOv2, GroundingDINO, zero-shot learning |
| 7. Advanced Topics | open-set recognition, novel species recognition, phylogentic reconstruction via clustering |
| 8. Hands-On Practice: Advanced Topics | finetuning, pollen detection/segmentation |

# Basics of Machine Learning via Pollen Recognition

- Given a training set of labeled pollen images

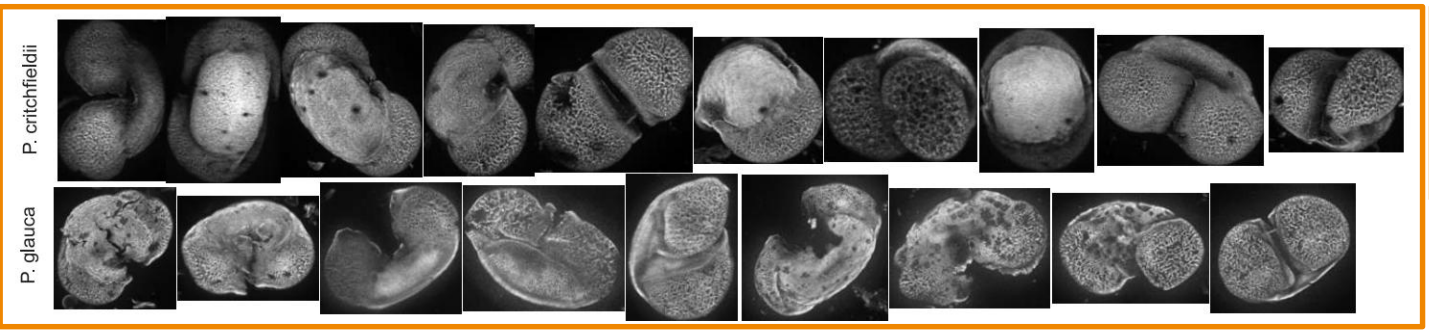



**a training set**

- data / features $x$
- targets $y \in \{1, 2, \ldots\}$

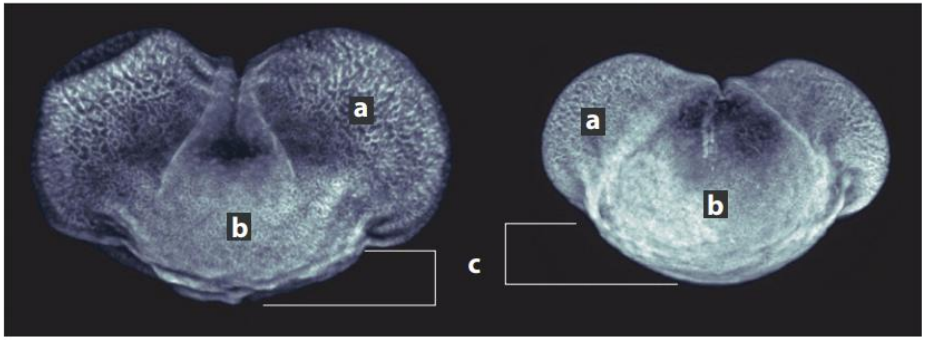
# Basics of Machine Learning via Pollen Recognition

- Given a training set of labeled pollen images




**a training set**

- data / features $x$
- targets $y \in \{1, 2, \ldots\}$

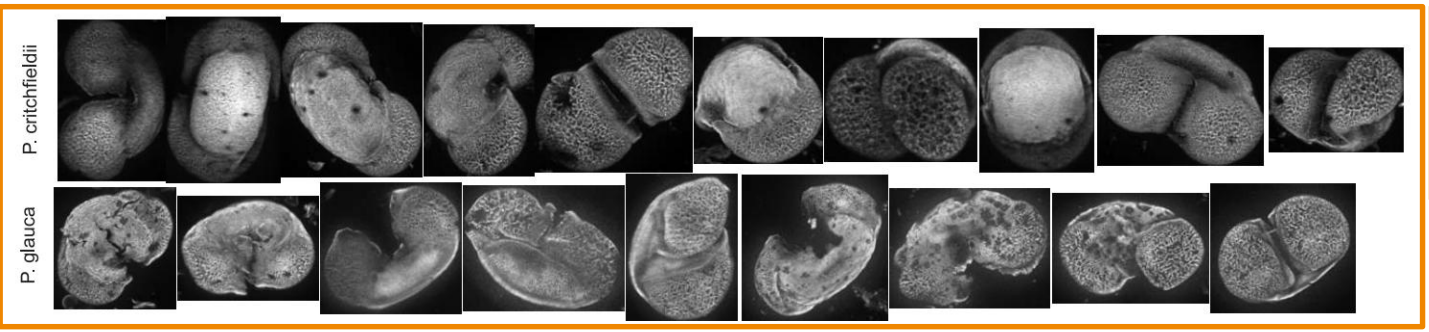



feature $x$:

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
……

# Basics of Machine Learning via Pollen Recognition

- Given a training set of labeled pollen images

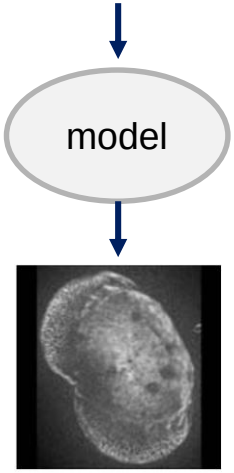



**a training set**

- data / features $x$
- targets $y \in \{1, 2, \ldots\}$

feature $x$:

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
……




$x'$

class-1 or class-2?

- Train a model (i.e., teach a machine) to recognize the species of a test image $x'$

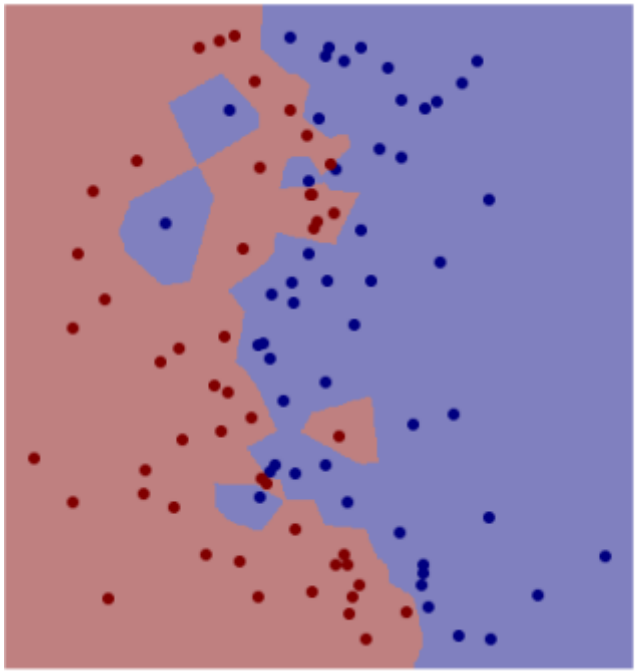
# Nearest Neighbor model

Find the nearest neighbor to $x'$ in the training dataset, e.g.,

1. rank training images according to their Euclidean distance to $x'$
2. select the training example that has the smallest distance to $x'$
3. return the corresponding class label as the prediction

**a training set**

- data / features $x$
- targets $y \in \{1, 2, \ldots\}$



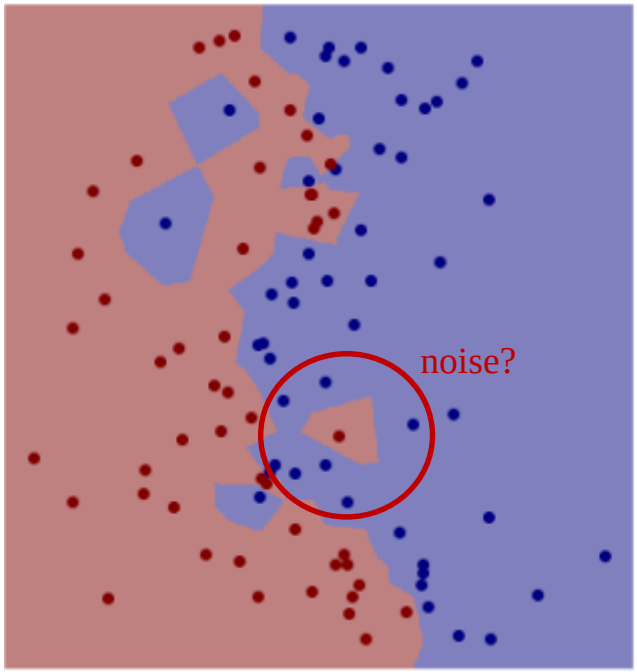
feature $x$:

size of (a)
shape of (a)
size of (b)
shape of (b)
size of (c)
shape of (c)
texture-1
texture-2
……

# Nearest Neighbor model

Find the nearest neighbor to *x'* in the training dataset, e.g.,

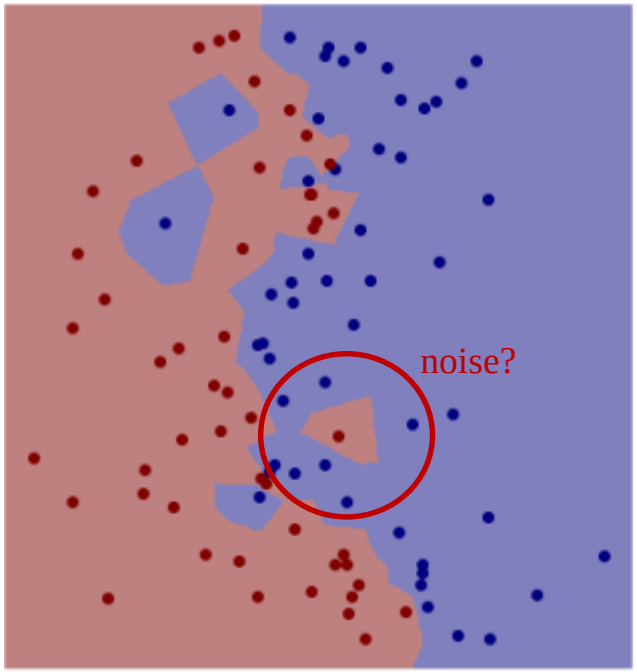
1. rank training images according to their Euclidean distance to *x'*
2. select the training example that has the smallest distance to *x'*
3. return the corresponding class label as the prediction



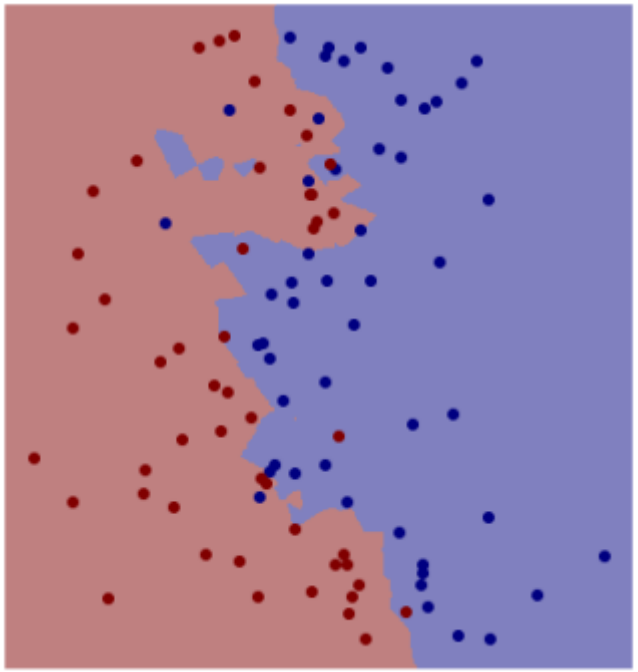
# *K*-Nearest Neighbor (KNN) model

Find the *K* nearest neighbor to *x'* in the training dataset, e.g.,

1. rank training images according to their Euclidean distance to *x'*
2. select *K* training examples that have the smallest distance to *x'*
3. vote for the predicted label among these *K* examples




*K=1*

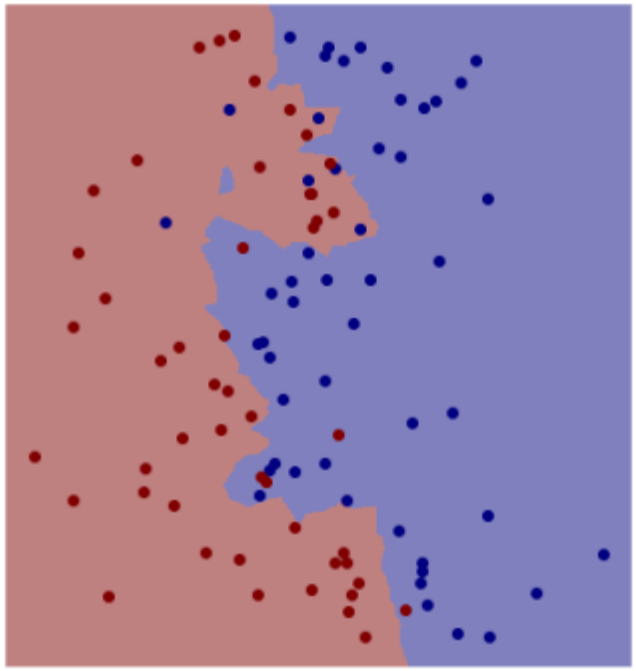


*K=3*

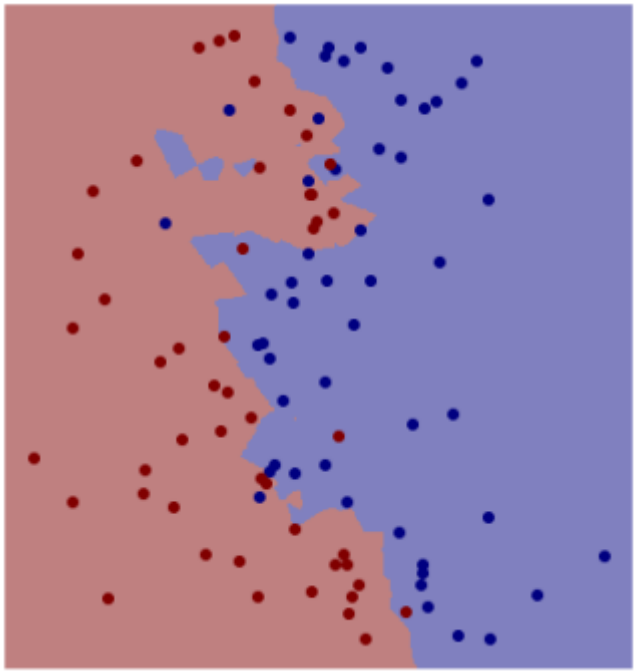
# *K*-Nearest Neighbor (KNN) model

Find the *K* nearest neighbor to *x'* in the training dataset, e.g.,

1. rank training images according to their Euclidean distance to *x'*
2. select *K* training examples that have the smallest distance to *x'*
3. vote for the predicted label among these *K* examples



*K=5*

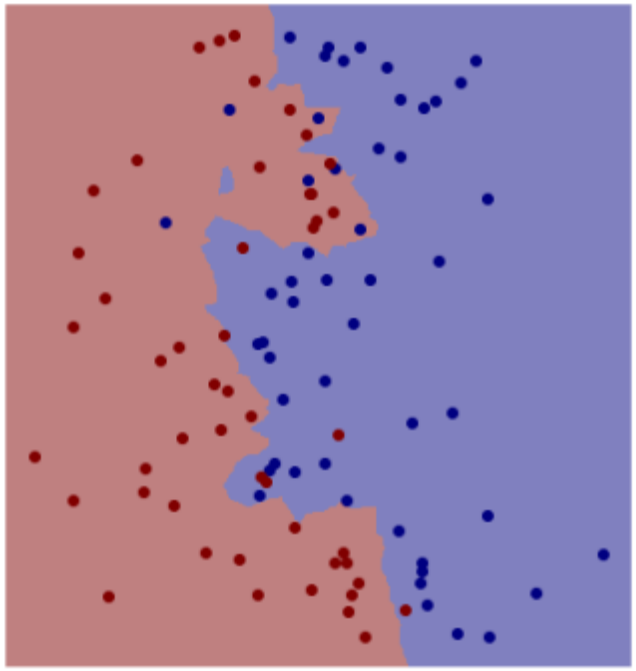


*K=3*

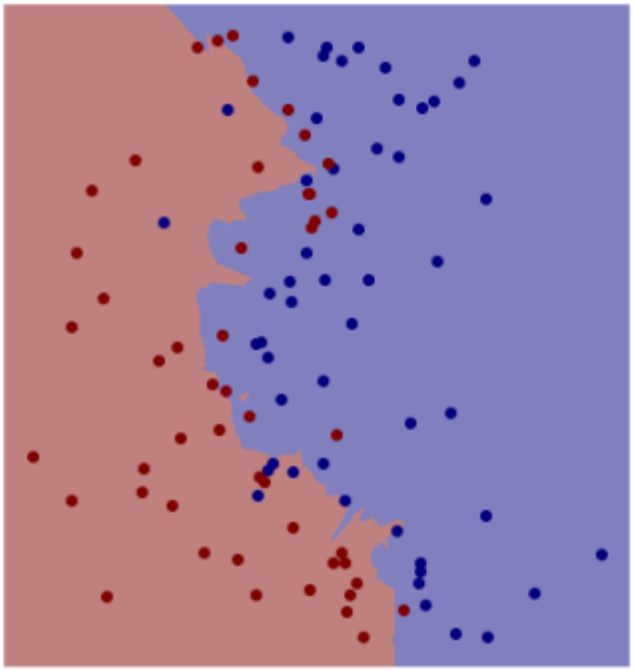
# *K*-Nearest Neighbor (KNN) model

Find the *K* nearest neighbor to *x'* in the training dataset, e.g.,

1. rank training images according to their Euclidean distance to *x'*
2. select *K* training examples that have the smallest distance to *x'*
3. vote for the predicted label among these *K* examples



*K=5*



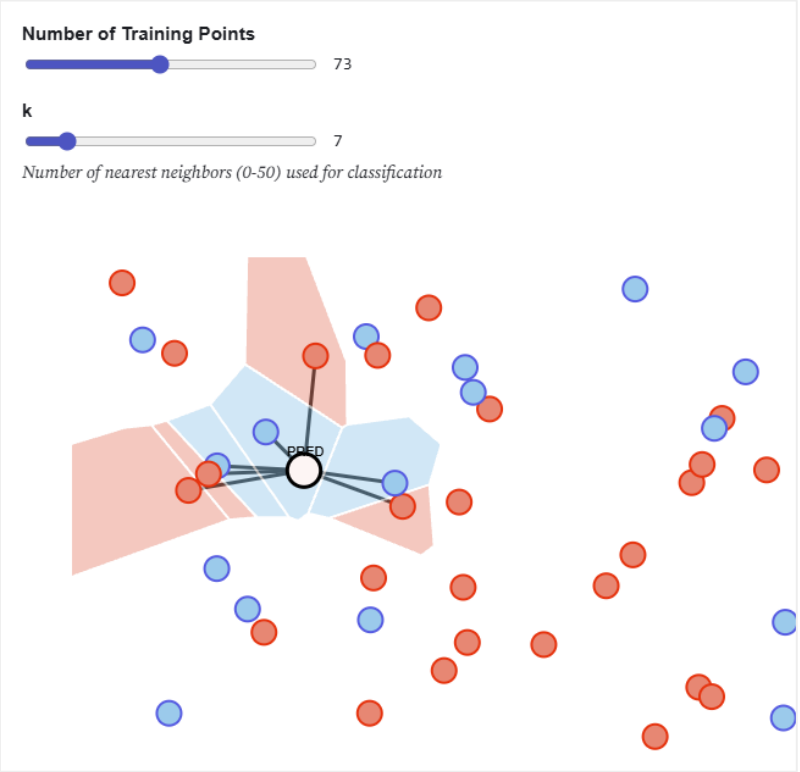
*K=20*

# *K*-Nearest Neighbor (KNN) model

Find the *K* nearest neighbor to *x'* in the training dataset, e.g.,

1. rank training images according to their Euclidean distance to *x'*
2. select *K* training examples that have the smallest distance to *x'*
3. vote for the predicted label among these *K* examples

Live demo: [https://observablehq.com/@jwilber/live-k-nearest-neighbors-classification](https://observablehq.com/@jwilber/live-k-nearest-neighbors-classification)

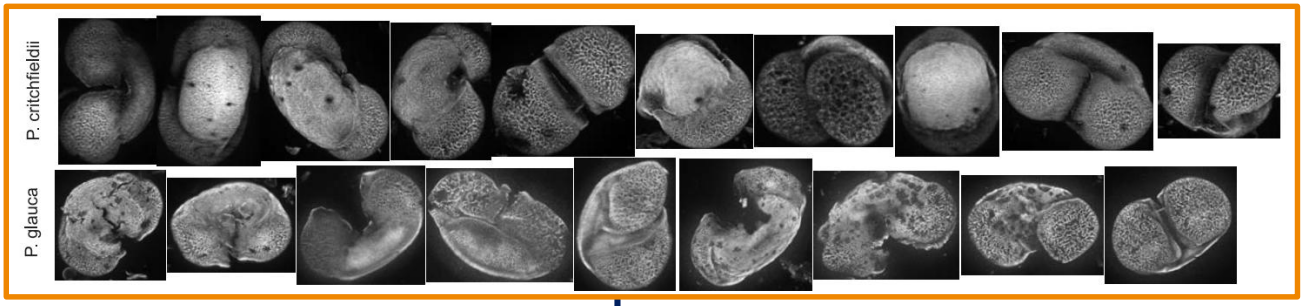


# *K*-Nearest Neighbor (KNN) model

Find the *K* nearest neighbor to *x'* in the training dataset, e.g.,

1. rank training images according to their Euclidean distance to *x'*
2. select *K* training examples that have the smallest distance to *x'*
3. vote for the predicted label among these *K* examples

KNN is a non-learned method. Its training is trivial: just store the training data as a database! Testing is about searching in the database to classify a test image.



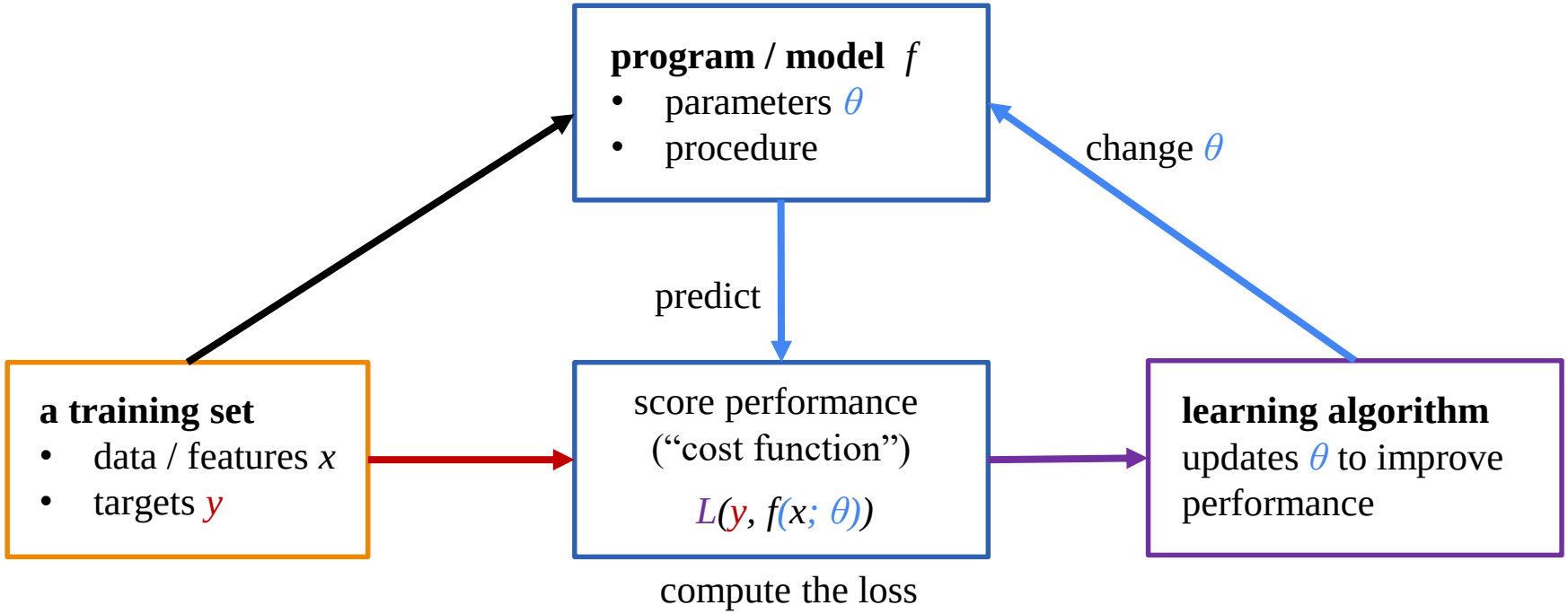

**a training set**

- data / features *x*
- targets *y*

model = training set

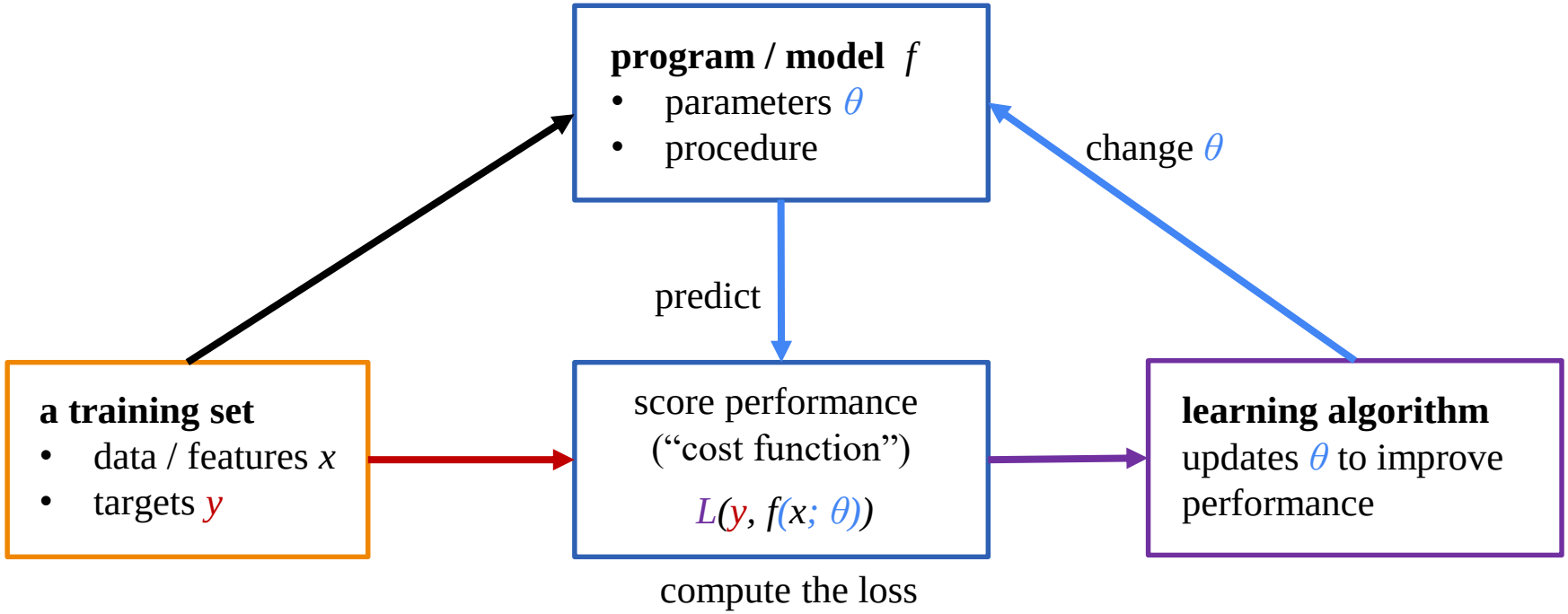
# An Overall Pipeline of Machine Learning

- Given a training set of labeled data
- Train a model to recognize the species of a test image

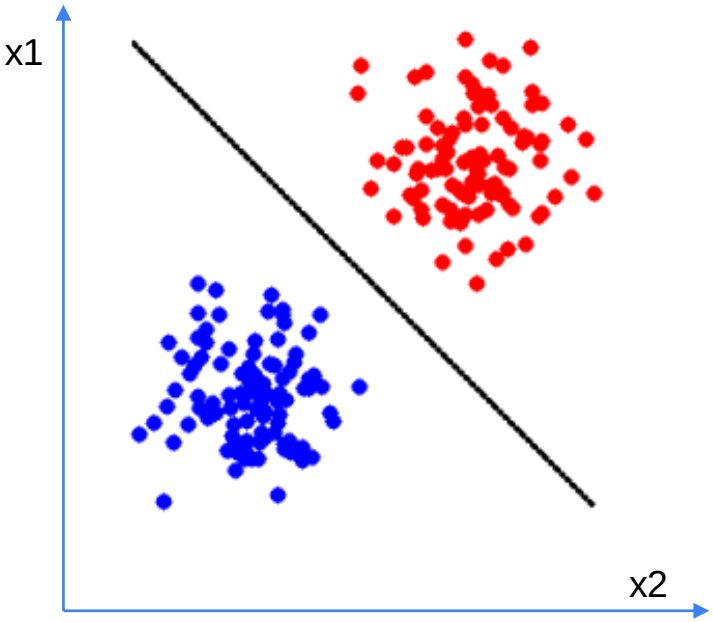


# Outline

- Nearest Neighbor
- **Linear Classification and Perceptron Learning**
- Logistic Regression and Gradient-Based Learning
- Multi-Class Classification

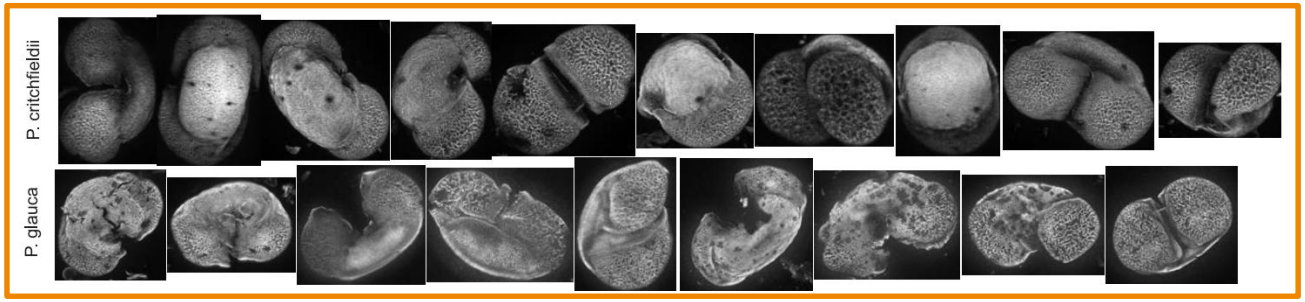


# Linear Classifier

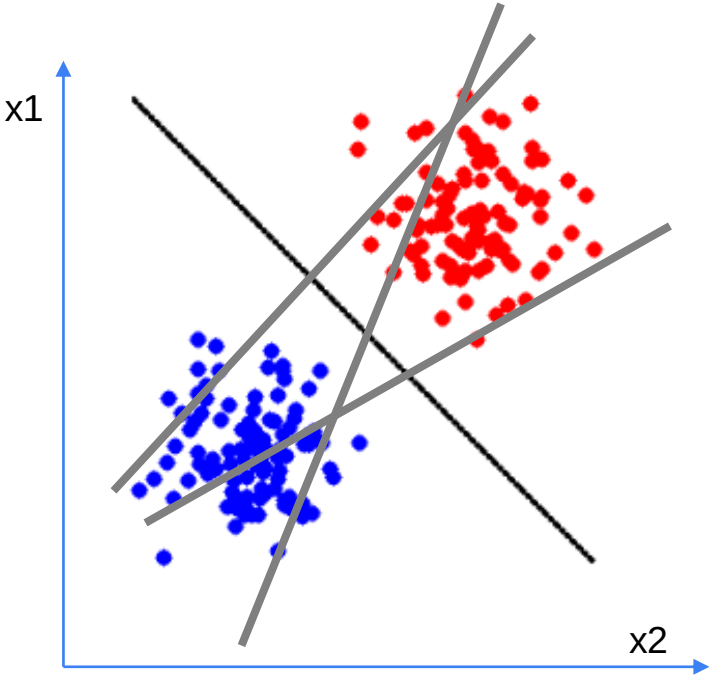



**a training set**

- data / features *x*
- targets *y*

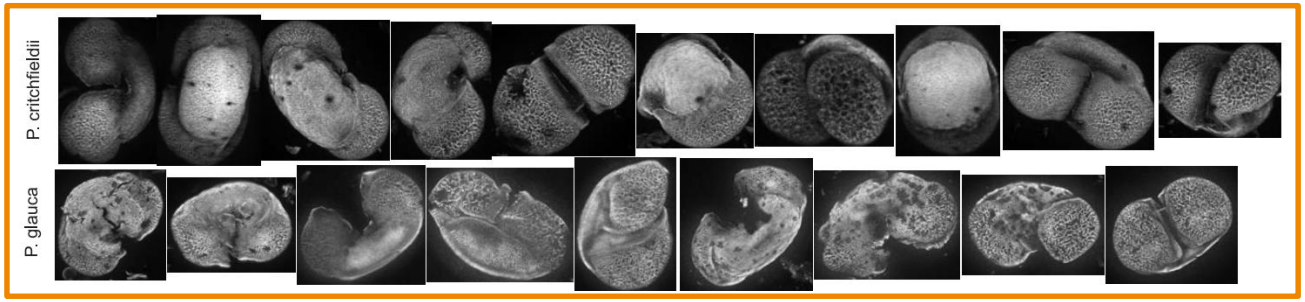


# Linear Classifier

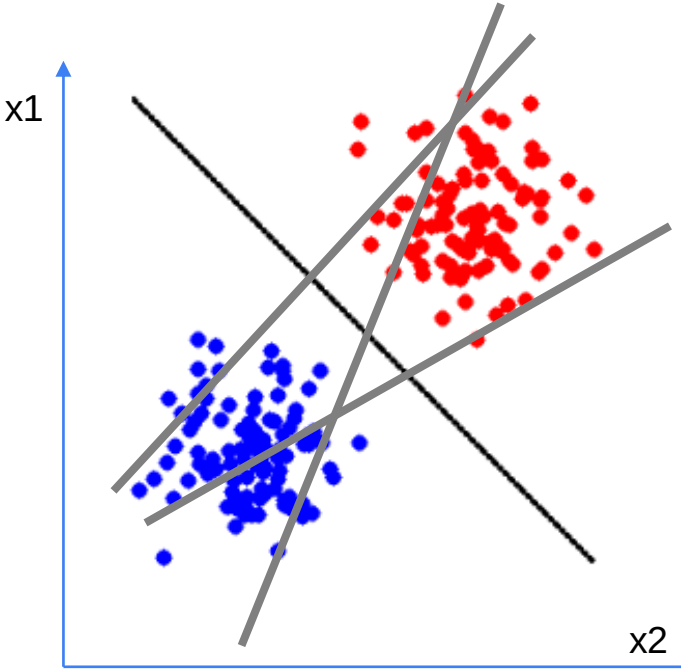


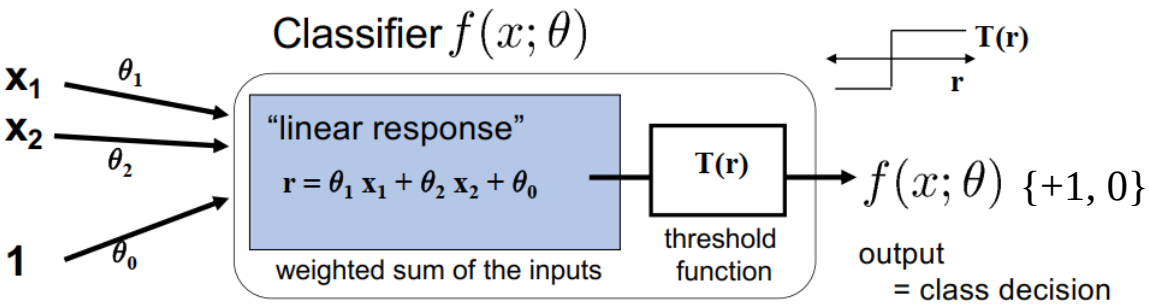

**a training set**

- data / features *x*
- targets *y*

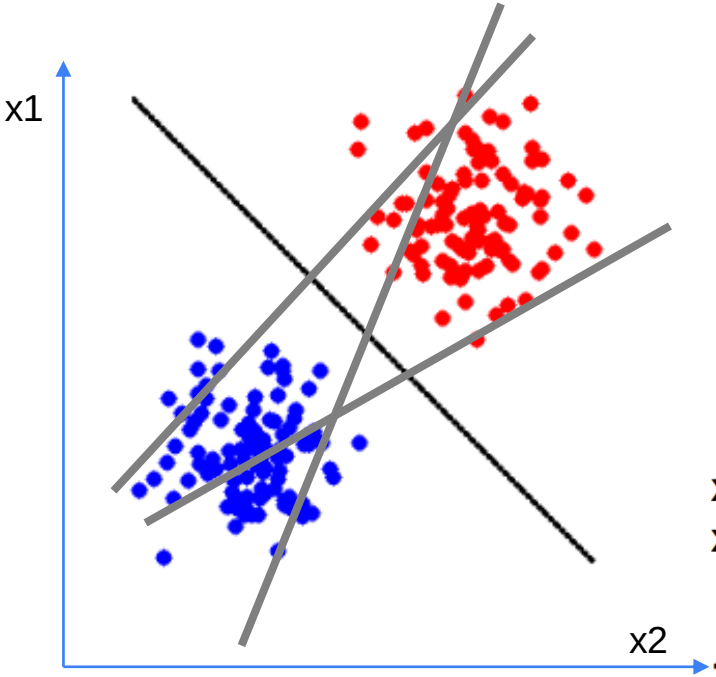


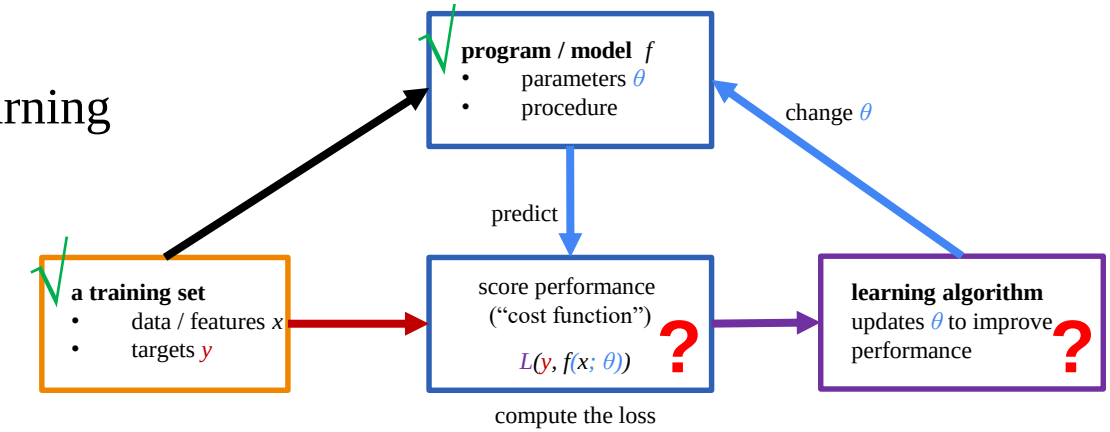
# Linear Classifier: Perceptron Learning

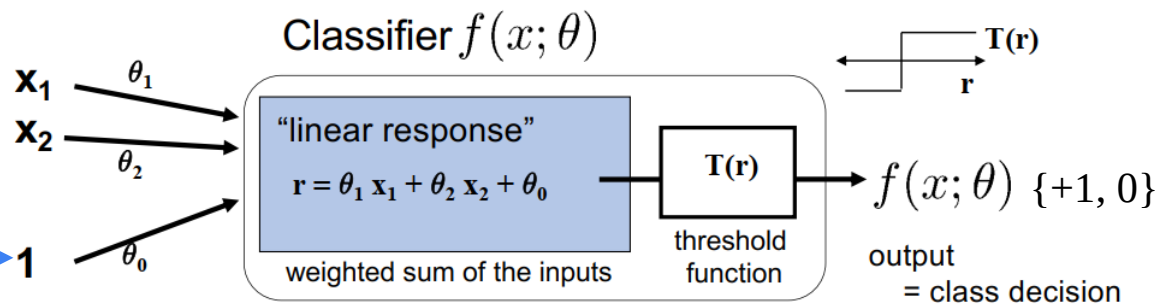


# Linear Classifier: Perceptron Learning




- Learning from Training Data:
  - training data = labeled feature vectors
  - Find parameter values that predict well (low error)
    - error is estimated on the training data
    - "true" error will be on future test data

- Define a loss function $J(\boldsymbol{\theta})$ :
  - Classifier error rate (for a given set of weights $\boldsymbol{\theta}$ and labeled data)

- Minimize this loss function (or, maximize accuracy)
  - An optimization or search problem over the vector $(\theta_0, \theta_1, \theta_2, \ldots)$

# Linear Classifier: Perceptron Learning

- How should we measure error?
  - Natural measure = "fraction we get wrong" (error rate)

$$\mathrm{err}(\theta) = \frac{1}{m}\sum_i \mathbb{1}\left[y^{(i)} \neq f(x^{(i)};\theta)\right] \quad \text{where} \quad \mathbb{1}\left[y \neq \hat{y}\right] = \begin{cases} 1 & y \neq \hat{y} \\ 0 & \text{o.w.} \end{cases}$$

# Linear Classifier: Perceptron Learning

- How should we measure error?
  - Natural measure = "fraction we get wrong" (error rate)

$$\mathrm{err}(\theta) = \frac{1}{m}\sum_i \mathbb{1}\left[y^{(i)} \neq f(x^{(i)};\theta)\right] \quad \text{where} \quad \mathbb{1}\left[y \neq \hat{y}\right] = \begin{cases} 1 & y \neq \hat{y} \\ 0 & \text{o.w.} \end{cases}$$

- Perceptron algorithm: an ~~SGD-like~~ algorithm

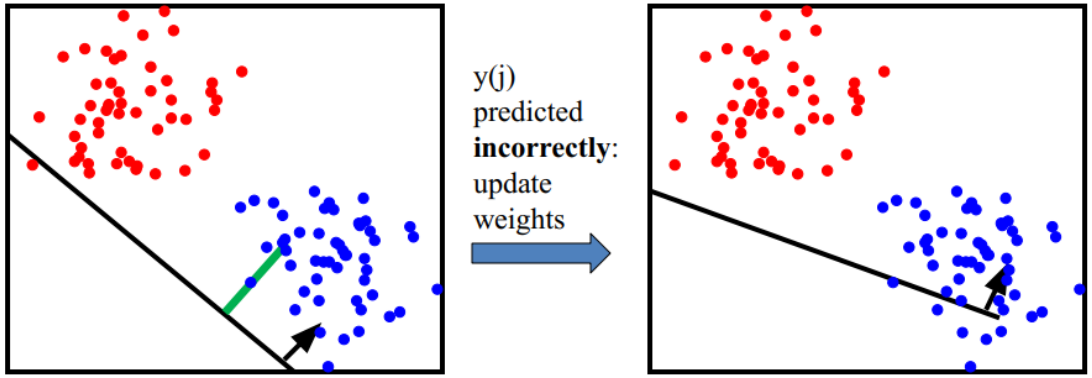
Freund, Y.; Schapire, R. E. (1999).

while ¬ done:

for each data point $j$:

$\hat{y}^{(j)} = \mathrm{sign}(\theta \cdot x^{(j)})$ (predict output for point j)

$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)}$ ("gradient-like" step)



# Linear Classifier: Perceptron Learning

- How should we measure error?
  - Natural measure = "fraction we get wrong" (error rate)

$$\mathrm{err}(\theta) = \frac{1}{m}\sum_i \mathbb{1}\left[y^{(i)} \neq f(x^{(i)};\theta)\right] \quad \text{where} \quad \mathbb{1}\left[y \neq \hat{y}\right] = \begin{cases} 1 & y \neq \hat{y} \\ 0 & \text{o.w.} \end{cases}$$

- Perceptron algorithm: an ~~SGD-like~~ algorithm

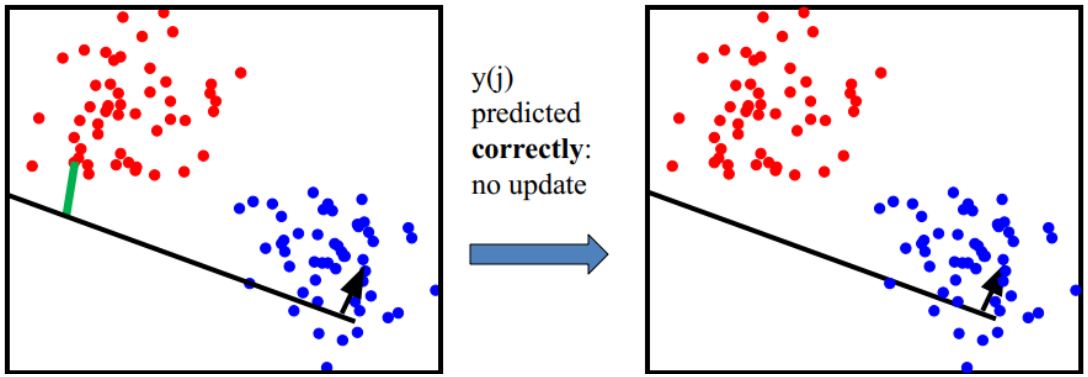
Freund, Y.; Schapire, R. E. (1999).

while ¬ done:

for each data point $j$:

$\hat{y}^{(j)} = \mathrm{sign}(\theta \cdot x^{(j)})$ (predict output for point j)

$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)}$ ("gradient-like" step)



# Linear Classifier: Perceptron Learning

- How should we measure error?
  - Natural measure = "fraction we get wrong" (error rate)

$$\mathrm{err}(\theta) = \frac{1}{m}\sum_i \mathbb{1}\left[y^{(i)} \neq f(x^{(i)};\theta)\right] \quad \text{where} \quad \mathbb{1}\left[y \neq \hat{y}\right] = \begin{cases} 1 & y \neq \hat{y} \\ 0 & \text{o.w.} \end{cases}$$

- Perceptron algorithm: an ~~SGD-like~~ algorithm

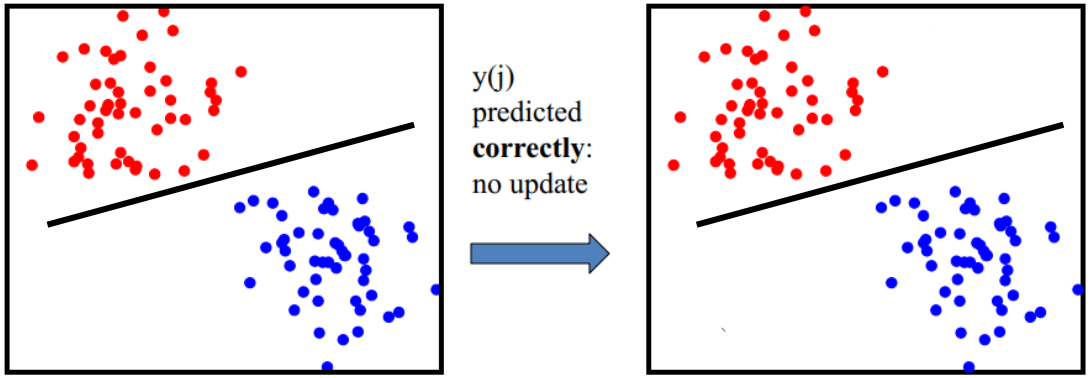
Freund, Y.; Schapire, R. E. (1999).

while ¬ done:

for each data point $j$:

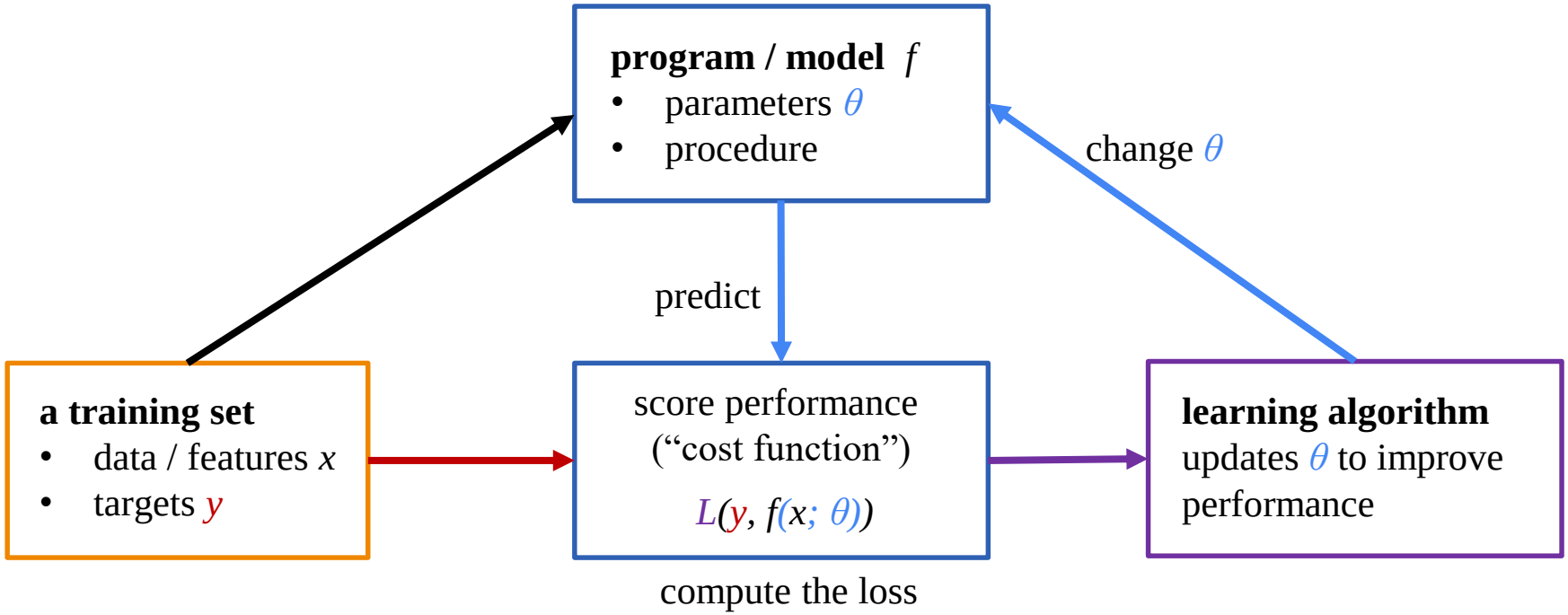
$\hat{y}^{(j)} = \mathrm{sign}(\theta \cdot x^{(j)})$ (predict output for point j)

$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)}$ ("gradient-like" step)



# Review the Pipeline of Machine Learning

- Given a training set of labeled data
- Train a model to recognize the species of a test image



# Perceptron Learning Algorithm

while $\neg$ done:

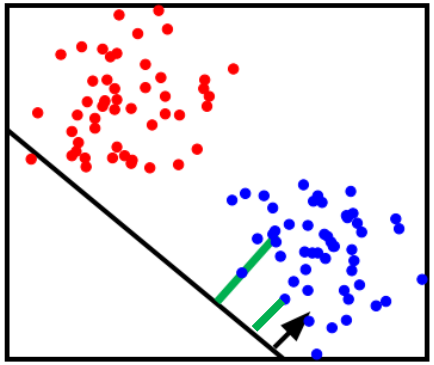
for each data point $j$:

$\hat{y}^{(j)} = \text{sign}(\theta \cdot x^{(j)})$ (predict output for point j)

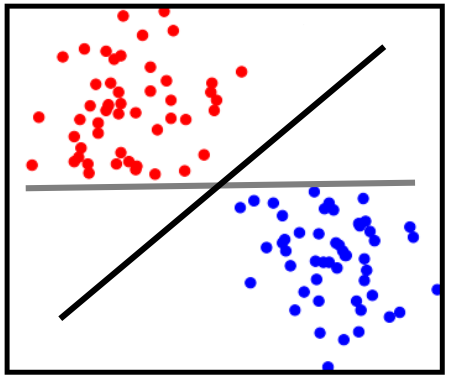
$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)}$ ("gradient-like" step)

## Issues

- It does not consider "more right / more wrong"

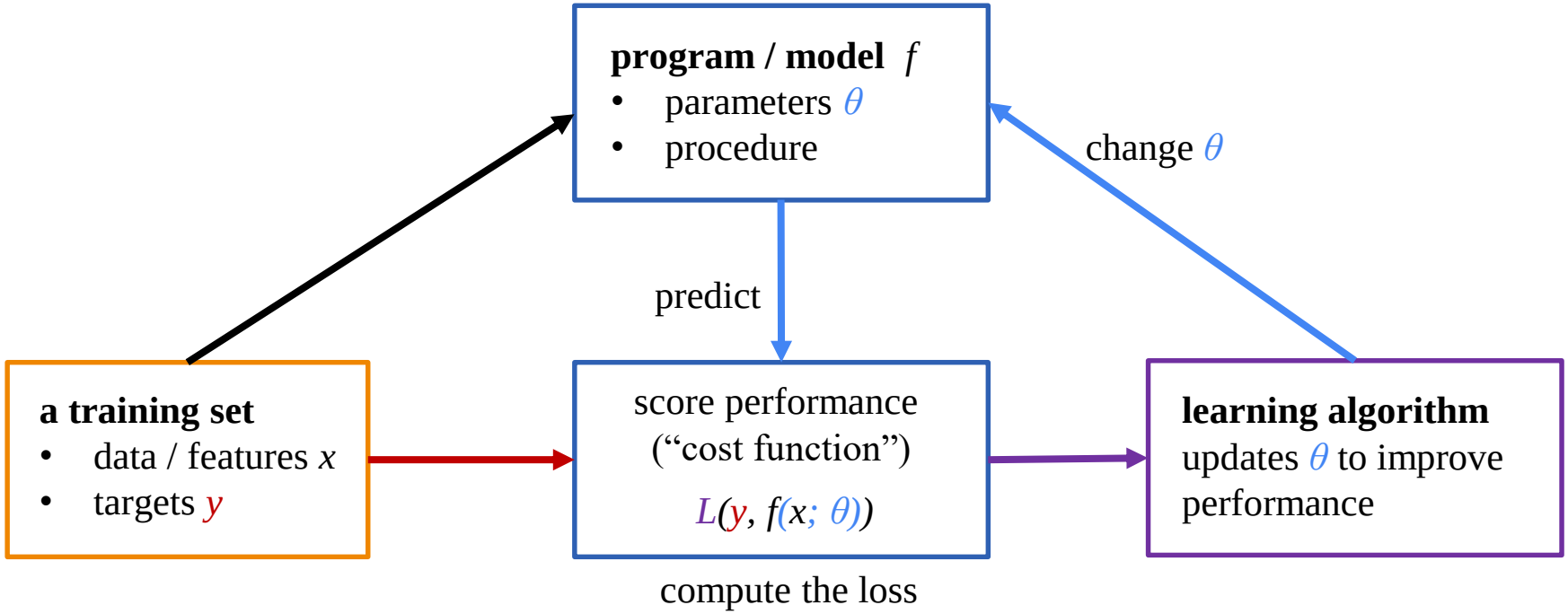


- It does not distinguish good vs. better classifiers



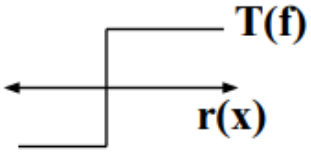
# Outline

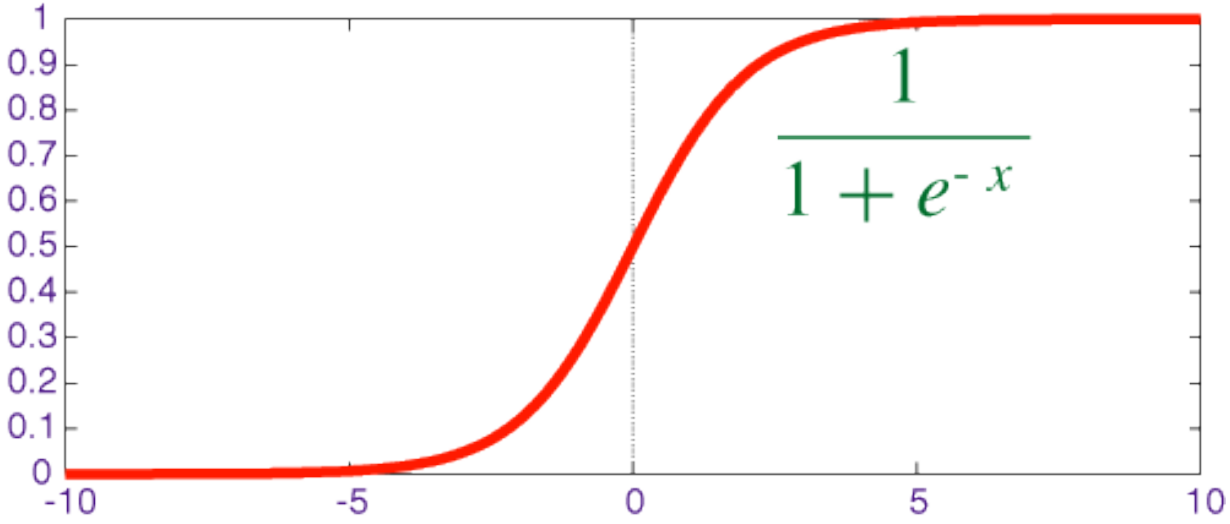
- Nearest Neighbor
- Linear Classification and Perceptron Learning
- **Logistic Regression and Gradient-Based Learning**
- Multi-Class Classification



# Surrogate Loss Functions

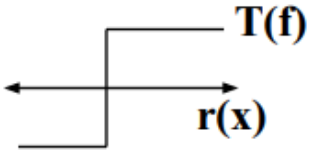
- Another solution: use a “smooth” loss
  - e.g., approximate the threshold function
  - Usually some smooth function of distance
    - Example: logistic “sigmoid”, looks like an “S”



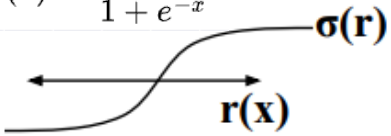



# Surrogate Loss Functions

- Another solution: use a "smooth" loss
  - e.g., approximate the threshold function
  - Usually some smooth function of distance
    - Example: logistic "sigmoid", looks like an "S"
  - Now, measure e.g. Mean Square Error (MSE)

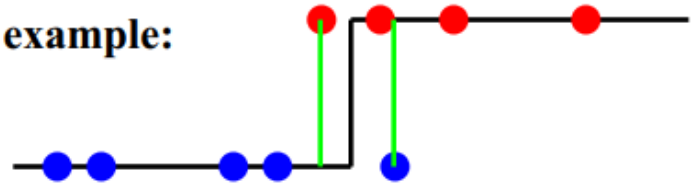



$$\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$$

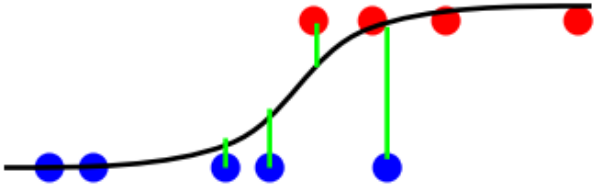



$$J(\underline{\theta}) = \frac{1}{m}\sum_j \left( \sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

**1D example:**



**Classification error = 2/9**



**MSE = $(0^2 + 1^2 + .2^2 + .25^2 + .05^2 + \ldots)/9$**

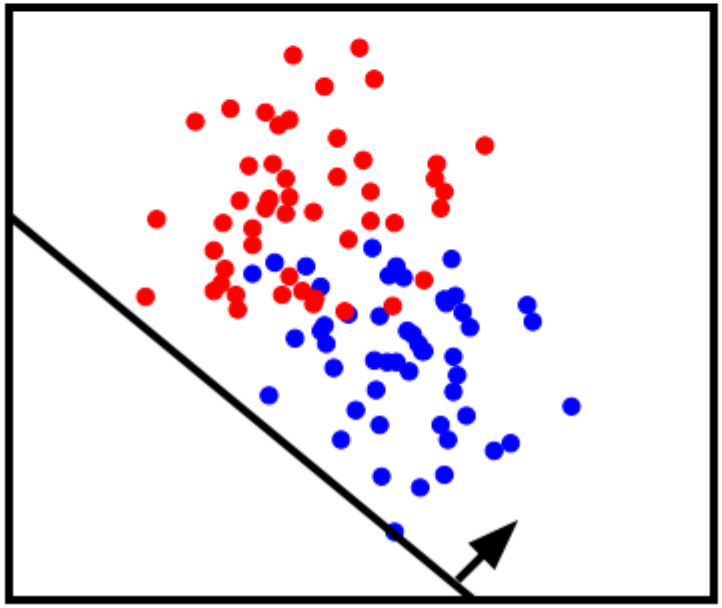
Training the Classifier

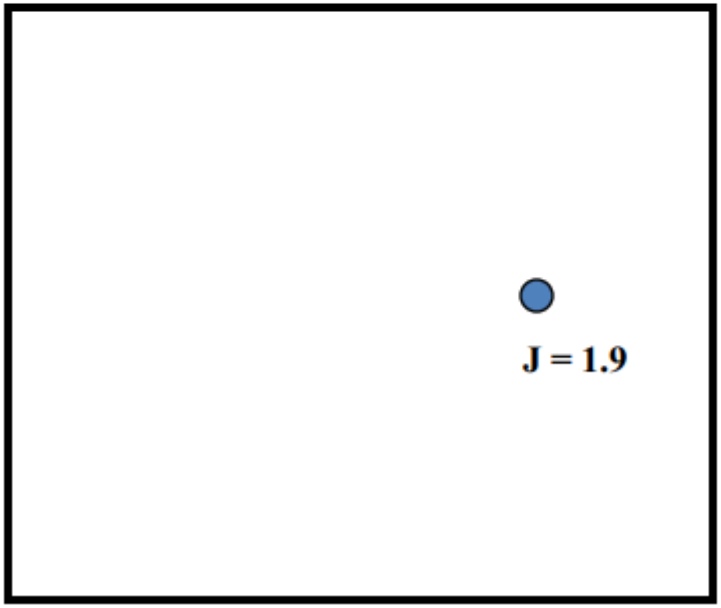
- Once we have a smooth measure of quality, we can find the "best" settings for the parameters of

$r(x_1,x_2) = a*x_1 + b*x_2 + c$

$$J(\underline{\theta}) = \frac{1}{m}\sum_j \left( \sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

- Example: 2D feature space ⇔ parameter space





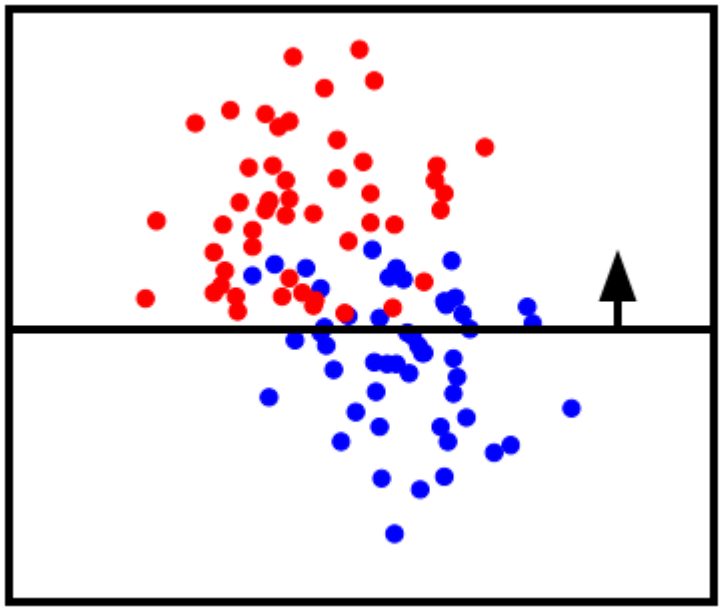
# Training the Classifier

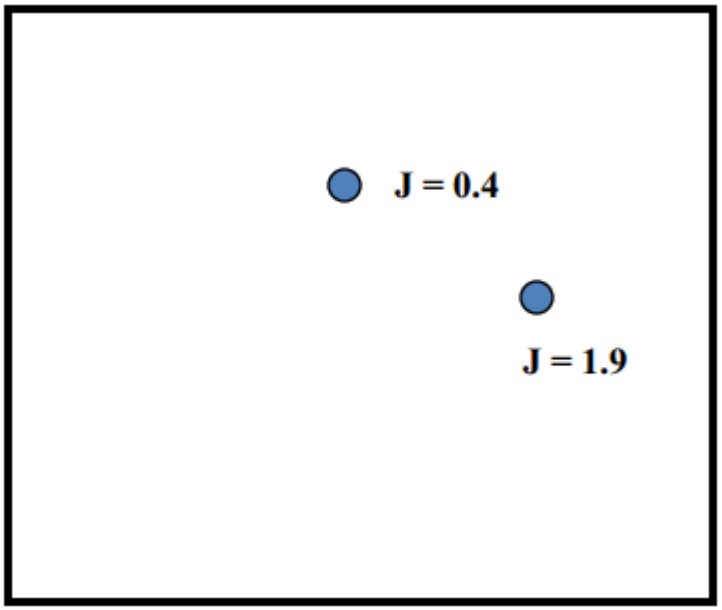
- Once we have a smooth measure of quality, we can find the "best" settings for the parameters of

$r(x_1,x_2) = a*x_1 + b*x_2 + c$

$$J(\underline{\theta}) = \frac{1}{m}\sum_j \left( \sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

- Example: 2D feature space ⇔ parameter space





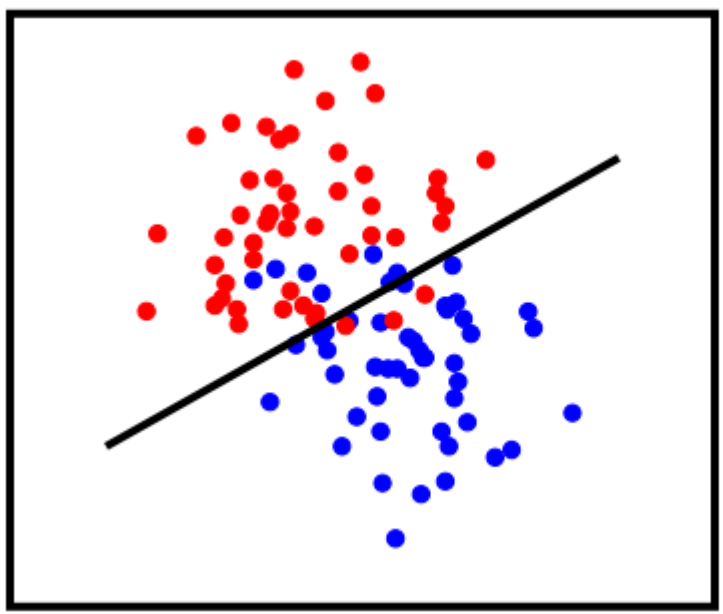
Training the Classifier

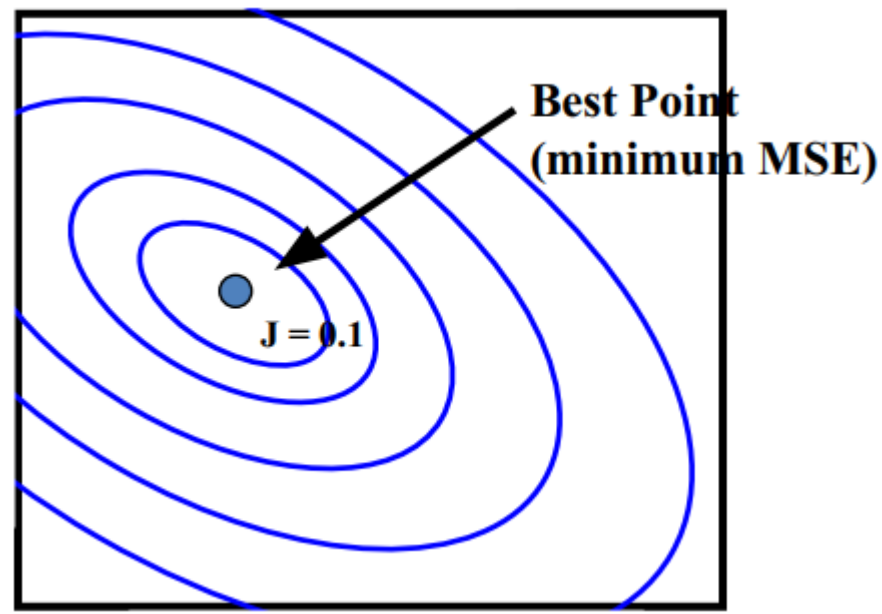
- Once we have a smooth measure of quality, we can find the "best" settings for the parameters of

$r(x_1,x_2) = a*x_1 + b*x_2 + c$

$$J(\underline{\theta}) = \frac{1}{m}\sum_j \left( \sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

- Example: 2D feature space ⇔ parameter space





# Finding the Best MSE

- Once we have a smooth measure of quality, we can find the "best" settings for the parameters of

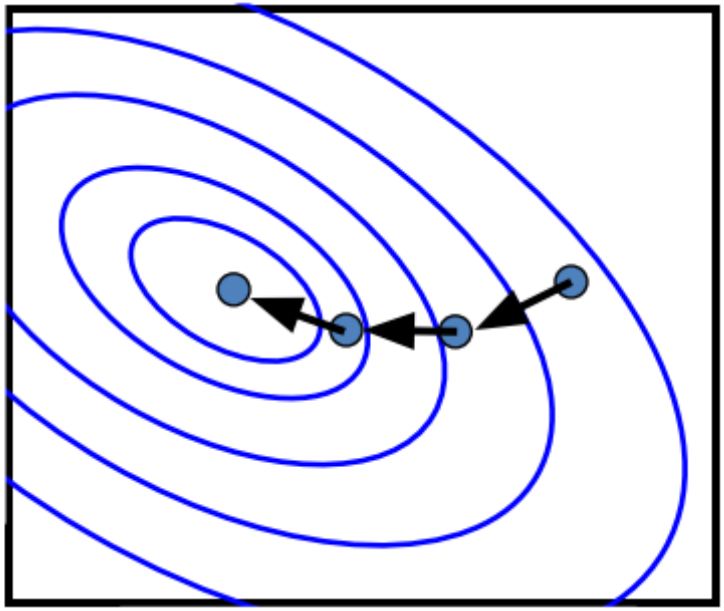
$r(x_1,x_2) = a*x_1 + b*x_2 + c$

$$J(\underline{\theta}) = \frac{1}{m}\sum_j \left( \sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

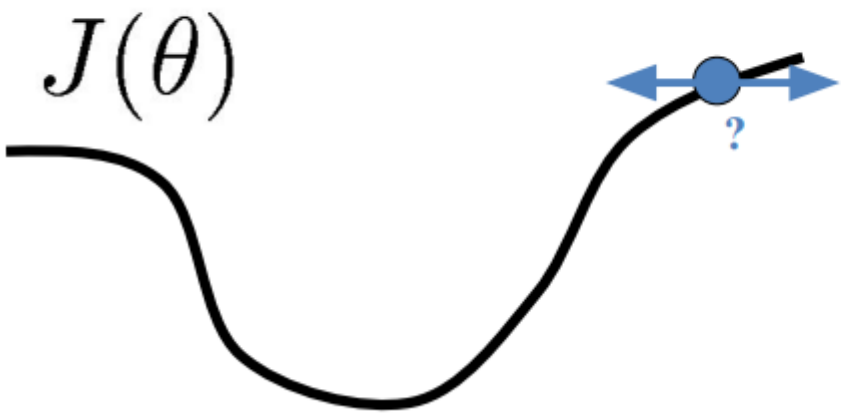
Methods:

- Gradient descent
  - Improve loss by small changes in parameters ("small" = learning rate)

**Gradient Descent**

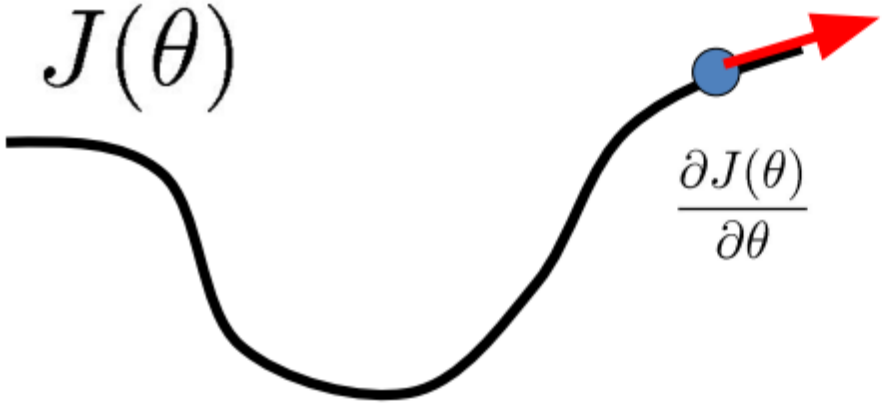


# Gradient Descent




- How to change θ to improve J(θ)?
- Choose a direction in which J(θ) is decreasing

Gradient Descent

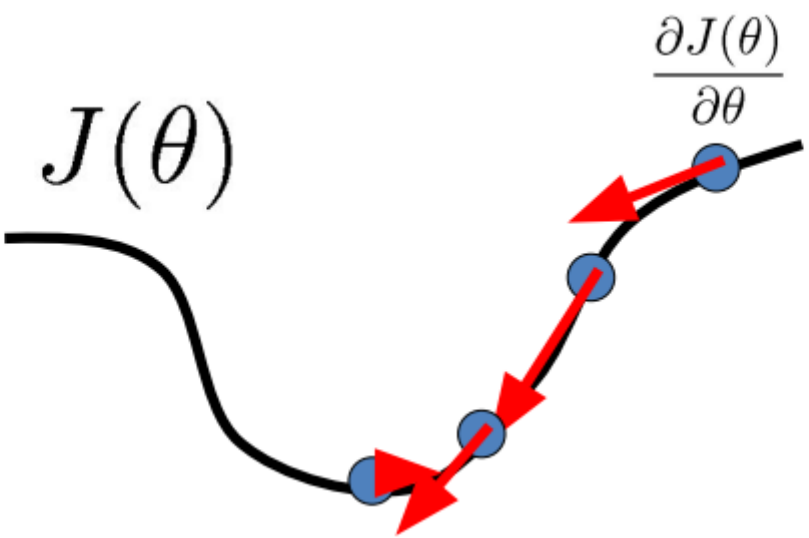



- How to change θ to improve J(θ)?
- Choose a direction in which J(θ) is decreasing
- Derivative $\frac{\partial J(\theta)}{\partial \theta}$
- Positive => increasing
- Negative => decreasing

# Gradient Descent

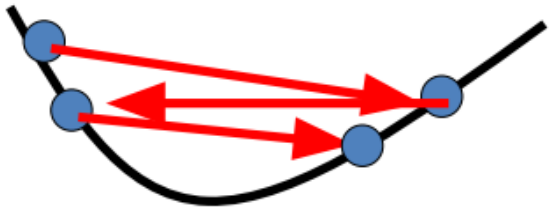
- Initialization
- Step size α
  - Can change over iterations
- Gradient direction
- Stopping condition

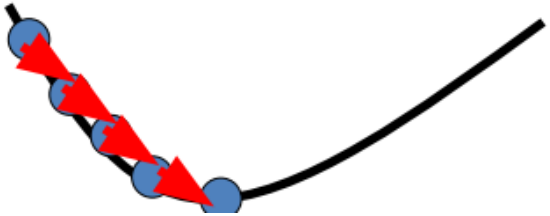
```
Initialize θ
Do{
 θ ← θ - α∇θJ(θ)
} while (α||∇θJ|| > ε)
```



# Comments on Gradient Descent

- Very general algorithm
- Step size
  - Too large? Too small? Automatic ways to choose?
  - May want step size to decrease with iteration

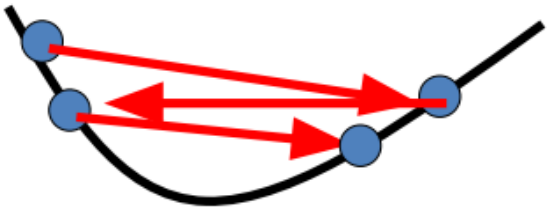


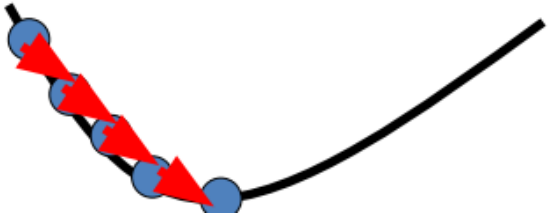


# Comments on Gradient Descent

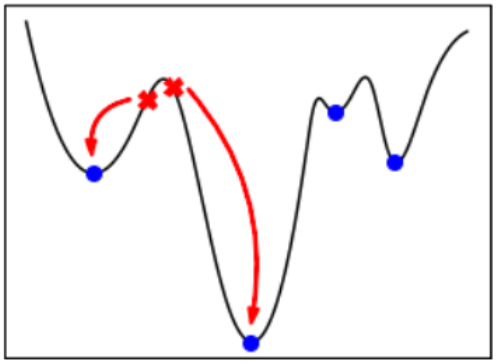
- Very general algorithm

- Step size
  - Too large? Too small? Automatic ways to choose?
  - May want step size to decrease with iteration

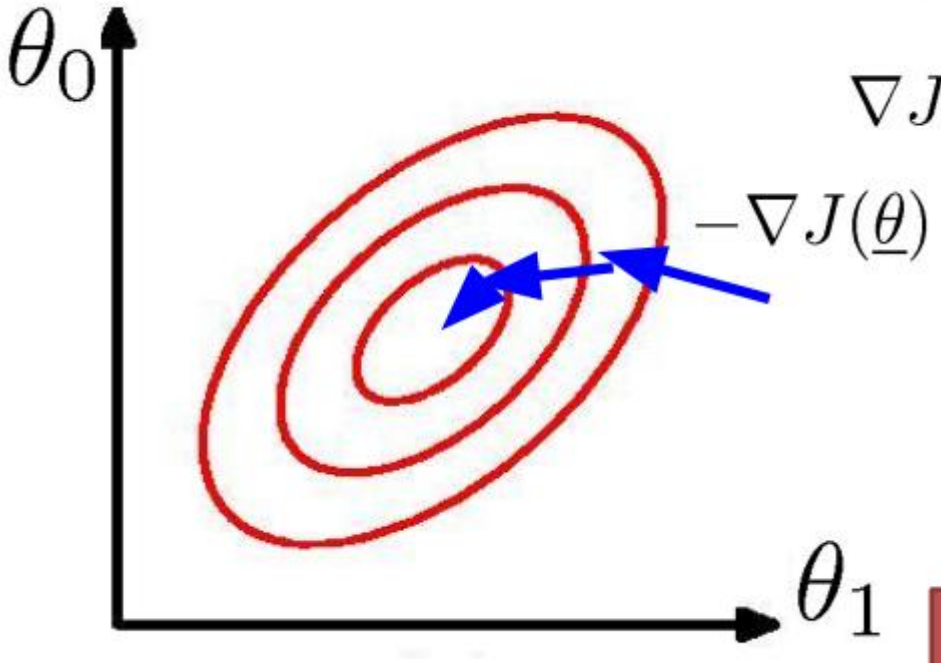




- Local minima
  - Sensitive to starting point



# Gradient Descent in More Dimensions




- Gradient vector

$$\nabla J(\underline{\theta}) = \left[ \frac{\partial J(\underline{\theta})}{\partial \theta_0} \quad \frac{\partial J(\underline{\theta})}{\partial \theta_1} \quad \cdots \right]$$

Indicates direction of steepest ascent
(negative = steepest descent)

Gradient Equations

- MSE (note, depends on function σ(.) )

$$J(\underline{\theta} = [a, b, c]) = \frac{1}{m} \sum_i ( \sigma(a x_1^{(i)} + b x_2^{(i)} + c) - y^{(i)} )^2$$

- What's the derivative with respect to one of the parameters?
  - Recall the chain rule of calculus:

$$\frac{\partial}{\partial a} f(g(h(a))) = f'(g(h(a)))\, g'(h(a))\, h'(a)$$

$$f(g) = (g)^2 \quad \Rightarrow \quad f'(g) = 2(g)$$

$$g(h) = \sigma(h) - y \quad \Rightarrow \quad g'(h) = \sigma'(h)$$

$$h(a) = a x_1^{(i)} + b x_2^{(i)} + c \quad \Rightarrow \quad h'(a) = x_1^{(i)}$$

w.r.t. b,c : similar; replace $x_1$ with $x_2$ or 1

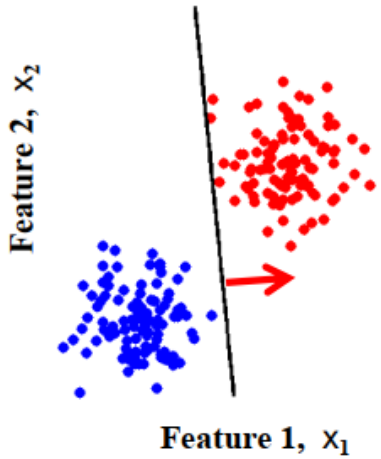
$$\frac{\partial J}{\partial a} = \frac{1}{m} \sum_i 2 \boxed{(\sigma(\theta \cdot x^{(i)}) - y^{(i)})} \boxed{\partial \sigma(\theta \cdot x^{(i)})\ x_1^{(i)}}$$

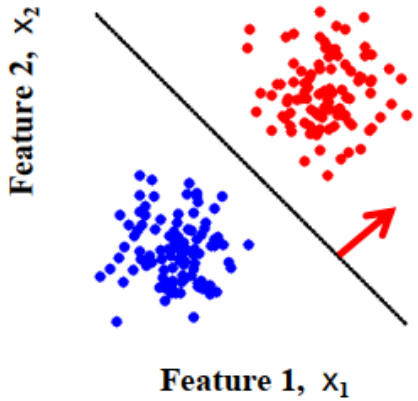
Error between class and prediction

Sensitivity of prediction to changes in parameter "a"

# Beyond Misclassification Rate

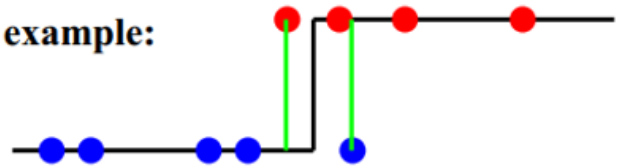
- Which decision boundary is "better"?
  - Both have zero training error (perfect training accuracy)
  - But, one of them seems intuitively better…



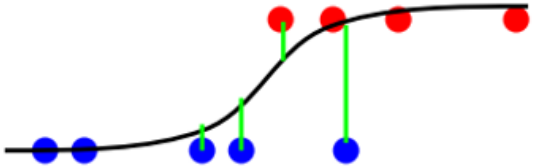




- Side benefit of many "smoothed" error functions
  - Encourages data to be far from the decision boundary
  - See more examples of this principle later...

**1D example:**



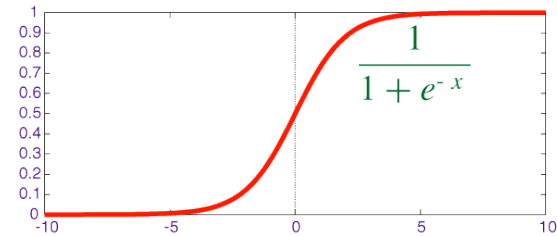
**Classification error = 2/9**



**MSE = $(0^2 + 1^2 + .2^2 + .25^2 + .05^2 + \ldots)/9$**

# Class Posterior Probabilities

$$p(y=1|x) = \text{sigmoid}(x) = \frac{1}{1+e^{-x}}$$

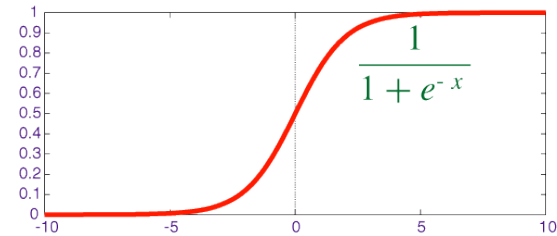



- Useful to also know class *probabilities*
- Some notation
  - p(y=0) , p(y=1) – class *prior* probabilities
    - How likely is each class in general?
  - p(x | y=c) – class conditional probabilities
    - How likely are observations "x" in that class?
  - p(y=c | x) – class posterior probability
    - How likely is class c *given* an observation x?

- We can compute posterior using Bayes' rule
  - p(y=c | x) = p(x|y=c) p(y=c) / p(x)
- Compute p(x) using sum rule / law of total prob.
  - p(x) = p(x|y=0) p(y=0) + p(x|y=1)p(y=1)

Class Posterior Probabilities

$$p(y=1|x) = \text{sigmoid}(x) = \frac{1}{1+e^{-x}}$$




- Consider comparing two classes
  - p(x | y=0) * p(y=0) vs p(x | y=1) * p(y=1)
  - Write probability of each class as
  - p(y=0 | x) = p(y=0, x) / p(x)
  - = p(y=0, x) / ( p(y=0,x) + p(y=1,x) )
  - = 1 / (1 + exp( -a ) ) (**)

  - a = log [ p(x|y=0) p(y=0) / p(x|y=1) p(y=1) ]
  - (**) called the logistic function, or logistic sigmoid.

$$\begin{aligned}
p(y=0|x) &= p(y=0,x) / p(x) \\
&= p(y=0,x) / (p(y=0,x)+p(y=1,x)) \\
&= 1 / (1 + p(y=1,x)/p(y=0,x)) \\
&= 1 / (1 + p(x|y=1)p(y=1)/p(x|y=0)p(y=0)) \\
&= 1 / (1 + exp( log(p(x|y=1)p(y=1)/p(x|y=0)p(y=0)) ) ) \\
&= 1 / (1 + exp( - log(p(x|y=0)p(y=0)/p(x|y=1)p(y=1)) ) )
\end{aligned}$$

# Logistic Regression

- Intepret $\sigma(\underline{\theta}\, x^T)$ as a probability that y = 1
- Use a negative log-likelihood loss function
  - If y = 1, cost is $-\log \Pr[y=1] = -\log \sigma(\underline{\theta}\, x^T)$
  - If y = 0, cost is $-\log \Pr[y=0] = -\log (1 - \sigma(\underline{\theta}\, x^T))$

- Can write this succinctly: i.e., the binary cross-entropy (BCE) loss

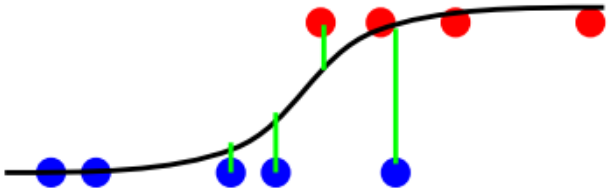
$$J(\underline{\theta}) = -\frac{1}{m}\left( \sum_i \underbrace{y^{(i)} \log \sigma(\theta \cdot x^{(i)})}_{\text{Nonzero only if y=1}} + \underbrace{(1-y^{(i)}) \log(1-\sigma(\theta \cdot x^{(i)}))}_{\text{Nonzero only if y=0}} \right)$$

# Logistic Regression

- Intepret $\sigma(\underline{\theta}\, x^T)$ as a probability that y = 1
- Use a negative log-likelihood loss function
  - If y = 1, cost is - log Pr[y=1] = - log $\sigma(\underline{\theta}\, x^T)$
  - If y = 0, cost is - log Pr[y=0] = - log (1 - $\sigma(\underline{\theta}\, x^T)$ )

- Can write this succinctly: i.e., the binary cross-entropy (BCE) loss

$$J(\underline{\theta}) = -\frac{1}{m}\left(\sum_i \underbrace{y^{(i)} \log \sigma(\theta \cdot x^{(i)})}_{\text{Nonzero only if y=1}} + \underbrace{(1-y^{(i)}) \log(1-\sigma(\theta \cdot x^{(i)}))}_{\text{Nonzero only if y=0}}\right)$$



NLL = - (log(.99) + log(.97) + ...)/9

# Gradient Equations

- Logistic neg-log likelihood loss:

  i.e., the binary cross-entropy (BCE) loss

$$J(\underline{\theta}) = -\frac{1}{m}\left(\sum_i y^{(i)} \log \sigma(\theta \cdot x^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\theta \cdot x^{(i)}))\right)$$

- What's the derivative with respect to one of the parameters?

$\theta = [\ldots, a, \ldots]$

$$\frac{\partial J}{\partial a} = -\frac{1}{m}\left(\sum_i y^{(i)} \frac{1}{\sigma(\theta \cdot x^{(i)})} \, \partial\sigma(\theta \cdot x^{(i)}) \; x_1^{(i)} + (1 - y(i)) \ldots\right)$$

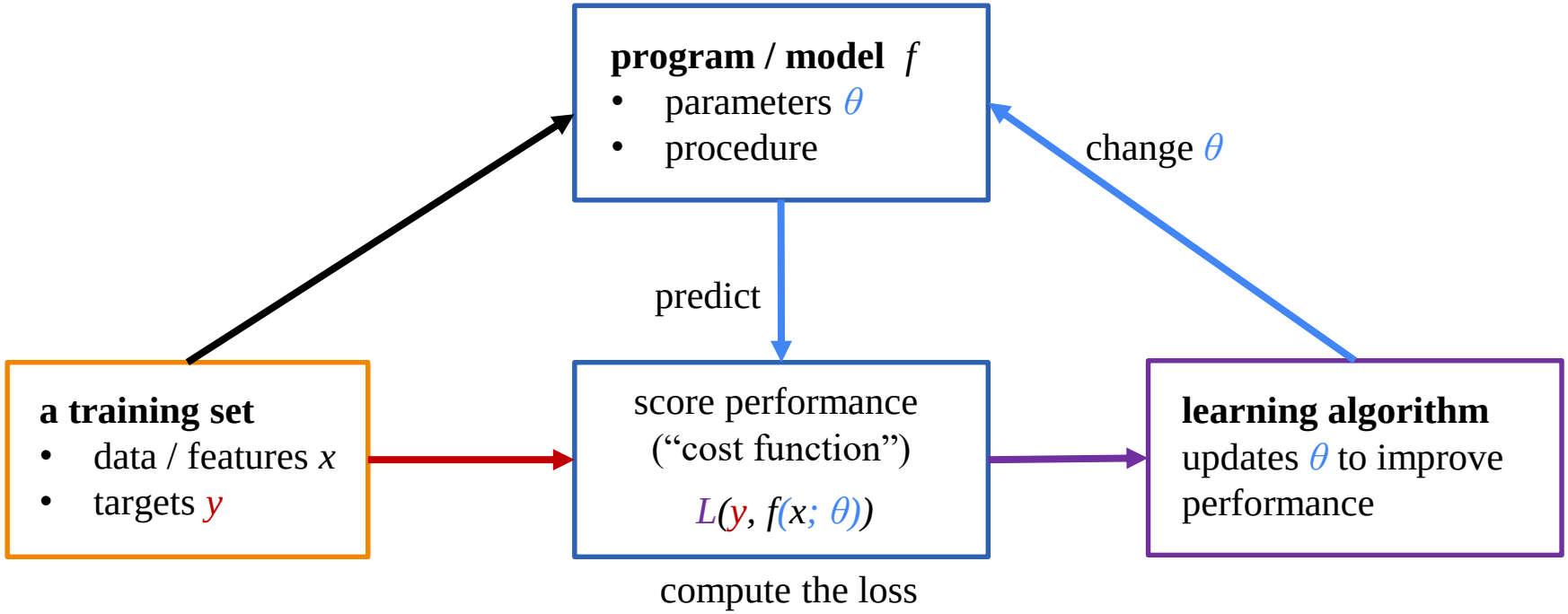
$$= -\frac{1}{m}\left(\sum_i y^{(i)} (1 - \sigma(\theta \cdot x^{(i)})) \; x_1^{(i)} + (1 - y^{(i)}) \ldots\right)$$

# Summary

- Linear classifier ⇔ perceptron
- Measuring quality of a decision boundary
  - Error rate (0/1 loss)
  - Logistic sigmoid + MSE criterion
  - Logistic Regression
- Learning the weights of a linear classifer from data
  - Reduces to an optimization problem
  - Perceptron algorithm
  - For MSE or Logistic NLL, we can do gradient descent
  - Gradient equations & update rules

# Outline

- Nearest Neighbor
- Linear Classification and Perceptron Learning
- Logistic Regression and Gradient-Based Learning
- **Multi-Class Classification**



# Multi-Class Linear Models

- What about multiple classes? One option:
  - Define one linear response per class
  - Choose class with the largest response

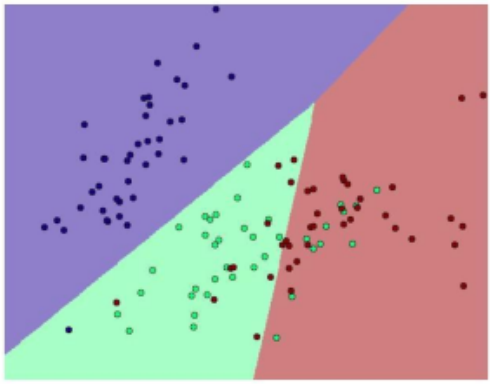
$$\theta = \begin{bmatrix} \theta_{00} & \cdots & \theta_{0n} \\ \vdots & \ddots & \vdots \\ \theta_{C0} & \cdots & \theta_{Cn} \end{bmatrix}$$

$$f(x;\theta) = \arg\max_c \ \theta_c \cdot x^T$$

  - Boundary between two classes, c vs. c'?

$$= \begin{cases} c & \text{if } \theta_c \cdot x^T > \theta_{c'} x^T \quad \Leftrightarrow (\theta_c - \theta_{c'}) x^T > 0 \\ c' & \text{otherwise} \end{cases}$$

- Linear boundary: $(\theta_c - \theta_{c'})\, x^T = 0$



# Multi-Class Linear Models

- Define the probability of each class:

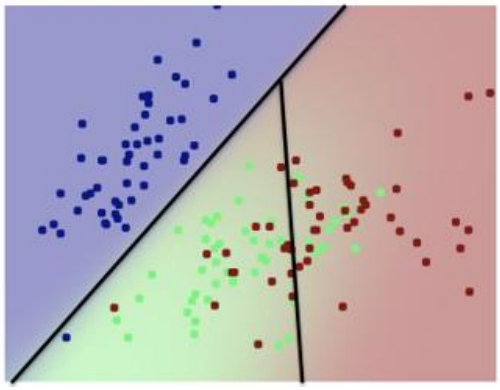
$$p(Y=y|X=x) = \frac{\exp(\theta_y \cdot x^T)}{\sum_c \exp(\theta_c \cdot x^T)}$$

(Y binary = logistic regression)

- Then, the NLL loss function is:

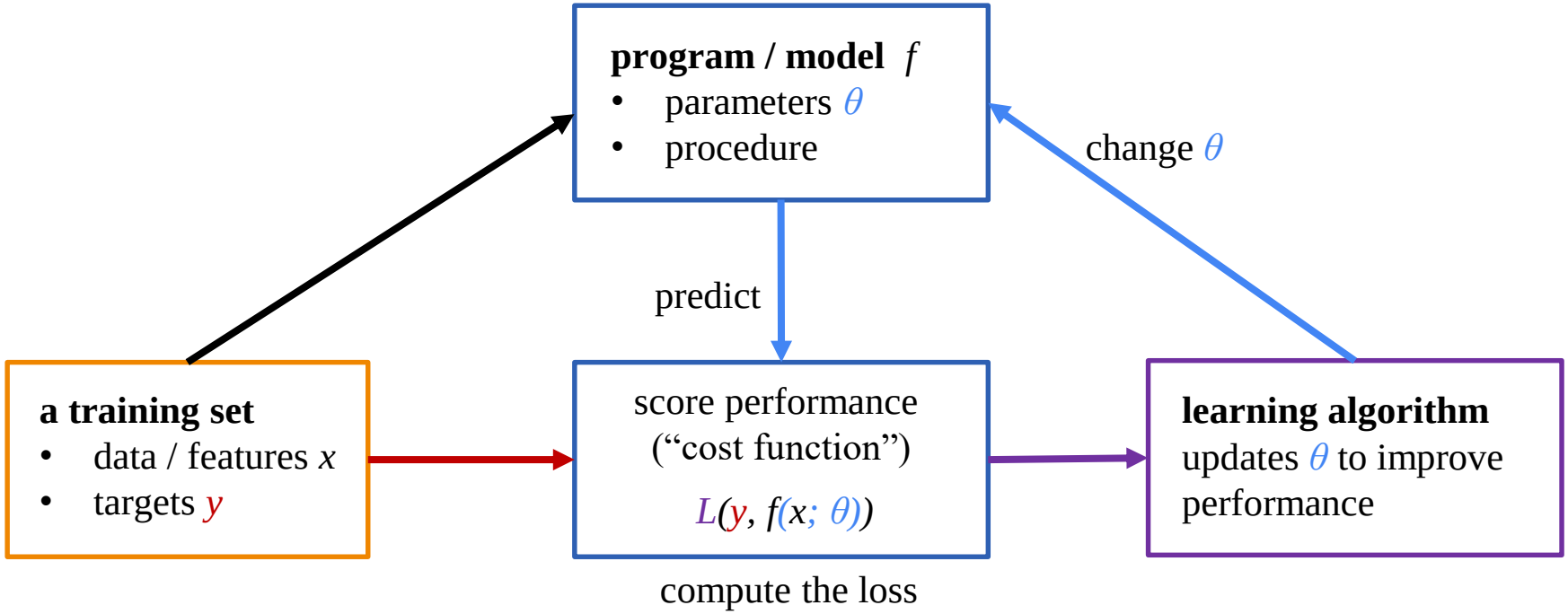
$$J(\theta) = -\frac{1}{m}\sum_i \log p(y^{(i)}|x^{(i)})$$

  - P: "confidence" of each class
    - Soft decision value
  - Decision: predict most probable
    - Linear decision boundary
  - Convex loss function



Recall binary CE loss

$$J(\underline{\theta}) = -\frac{1}{m}\Big(\sum_i \underbrace{y^{(i)} \log \sigma(\theta \cdot x^{(i)})}_{\text{Nonzero only if y=1}} + \underbrace{(1-y^{(i)}) \log(1-\sigma(\theta \cdot x^{(i)}))}_{\text{Nonzero only if y=0}}\Big)$$

# Review

- Nearest Neighbor
- Linear Classification and Perceptron Learning
- Logistic Regression and Gradient-Based Learning
- Multi-Class Classification